

RELATÓRIO TÉCNICO FINAL

Sprint 4 - App do Prestador e Integração Final

Projeto: FastDelivery - plataforma de delivery generalista

Disciplina: Laboratorio de Desenvolvimento de Aplicações Móveis e Distribuídas - PUC Minas

Data de validação: 29/06/2026

Fonte oficial: Projeto_LAMD_60445_2026_1.pdf, seção 3.5

Escopo: Aplicativo Flutter do prestador, fluxo cliente-MOM-prestador e notificação assíncrona

Resumo executivo

A Sprint 4 completa o FastDelivery com um app Flutter independente para o prestador, integrado ao backend Flask REST e ao RabbitMQ. O fluxo validado foi: cliente cria uma entrega no app, backend publica evento no MOM, ponte RabbitMQ -> WebSocket notifica o prestador em tempo real, prestador aceita a solicitação e o cliente reflete o status por polling.

1. Requisitos oficiais e atendimento

A seção 3.5 do edital define que a Sprint 4 deve entregar o app do prestador, notificação assíncrona, fluxo completo ponta a ponta, screencast, relatório técnico final e repositório organizado. A matriz abaixo resume o atendimento verificado.

Critério Sprint 4	Peso	Atendimento e evidência
Funcionalidade do app do prestador	25%	Atendido. App Flutter independente com telas Pendentes, Detalhe e Em andamento.
Fluxo completo ponta a ponta	30%	Atendido. Cliente criou entrega #2, prestador aceitou, cliente refletiu status aceito.
Notificação assíncrona ao prestador	20%	Atendido. Backend usa RabbitMQ -> fila fastdelivery.realtime -> WebSocket /ws/eventos.
Qualidade e clareza do screencast	10%	Roteiro derivado dos prints deste relatório; o video deve acompanhar a entrega final.
Relatório técnico final	15%	Atendido neste documento, com arquitetura, decisões, dificuldades, reflexao e referencias.

2. Arquitetura implementada

A arquitetura preserva as sprints anteriores e adiciona apenas os componentes necessários para notificação em tempo real ao prestador. O backend Flask continua expondo REST, persistindo em SQLite e publicando eventos de domínio no RabbitMQ. O worker histórico continua separado do backend, provando independência entre produtor e consumidor.

Componentes principais:

- App cliente Flutter: cria, lista e acompanha entregas via REST, com polling de 5 segundos.
- Backend Flask: endpoints REST, casos de uso, produtor de eventos e ponte WebSocket.
- SQLite: banco local entregas.db inicializado pelo backend para entregas e histórico processado.
- RabbitMQ em Docker: exchange topic fastdelivery.events, fila durável fastdelivery.entregas, DLQ e fila realtime.
- consumer_worker.py: consumidor standalone da fila fastdelivery.entregas, sem Flask e sem REST.
- App prestador Flutter: consome REST para sincronização e WebSocket /ws/eventos para push de eventos.

Fluxo de eventos

Cliente -> POST /entregas -> EntregaUseCases publica entrega.criada -> exchange fastdelivery.events -> fila fastdelivery.entregas para histórico e fila fastdelivery.realtime para ponte WebSocket -> app prestador recebe evento -> PATCH /entregas/<id>/status -> entrega.status_atualizado -> cliente atualiza por polling.

Fila/rota	Papel	Característica
fastdelivery.events	Exchange topic do MOM	Centraliza os eventos entrega.criada e entrega.status_atualizado.
fastdelivery.entregas	Fila do consumer_worker.py	Durável, com DLX/DLQ; usada para histórico processado.
fastdelivery.realtime	Fila da ponte WebSocket	Dedicada ao tempo real; evita competing consumers com o worker.
/ws/eventos	Canal do prestador	WebSocket backend -> app prestador, com keepalive e reconexão no app.

3. Decisões de design

- O app do prestador foi criado como projeto separado, com applicationId distinto e camadas equivalentes ao app cliente.
- As ações do prestador reutilizam PATCH /entregas/<id>/status, sem criar endpoints novos para aceitar, recusar, iniciar trânsito ou concluir.

- A ação Recusar foi mapeada para o status cancelado, pois o contrato de domínio existente não define um status recusado.
- A notificação real-time não expõe RabbitMQ ao mobile; o broker fica atrás do backend, que entrega eventos por WebSocket.
- A fila fastdelivery.realtime é separada da fila fastdelivery.entregas para que worker histórico e ponte recebam cópias independentes dos eventos.
- O realtime aceita perda pontual de notificação ao vivo porque o app faz re-sincronização REST ao conectar ou reconectar.

4. App do prestador

O app do prestador atende ao mínimo de três telas solicitado. A tela Pendentes lista entregas com status pendente e recebe novas solicitações em tempo real. A tela Detalhe mostra dados completos e ações contextuais. A tela Em andamento reúne entregas aceitas ou em trânsito.

Tela	Arquivo principal	Função
Pendentes	presentation/screens/pendentes_screen.dart	Lista solicitações pendentes e mostra novas entradas recebidas por WebSocket.
Detalhe	presentation/screens/detalhe_screen.dart	Exibe dados, timeline e botões Aceitar/Recusar ou avançar status.
Em andamento	presentation/screens/andamento_screen.dart	Acompanha entregas aceitas/em trânsito e atualiza por eventos.

A camada data concentra EntregaApiService para REST e EventosRealtimeService para WebSocket. A camada application usa controllers ChangeNotifier, mantendo estado, carregamento, erros, reconexão e re-sync fora das telas.

5. Evidências de execução no Chrome

A validação foi executada em 29/06/2026 com RabbitMQ em Docker, backend Flask na porta 5055, worker independente e os dois apps Flutter servidos como web build local no Google Chrome. Como o backend REST foi planejado para mobile/emulador e não define CORS para web, a instância local do Chrome usada para os prints foi aberta com segurança web desativada apenas para esta verificação local. Isso não altera o código do projeto.

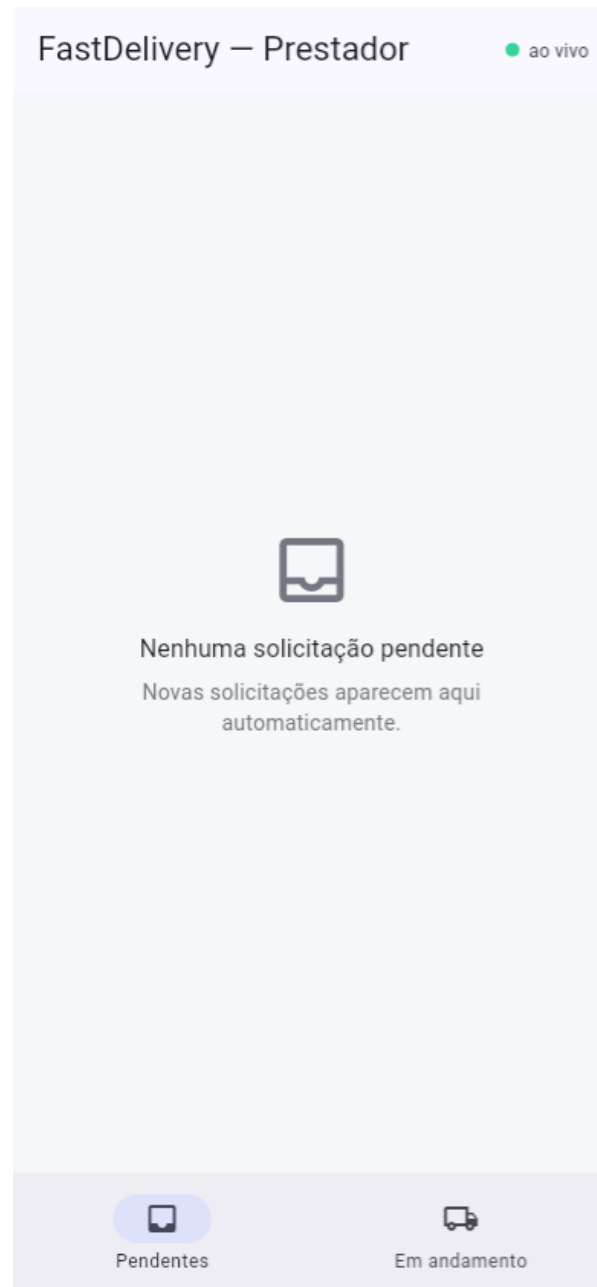


Figura 1 - App prestador conectado ao WebSocket, sem solicitações pendentes.



← Nova entrega

Descrição
📄 Sprint 4 - WS ao vivo

Origem
📍 PUC Minas - Portaria 1

Destino
📍 Centro de Distribuicao FastDelivery

➤ Criar entrega

Figura 2 - App cliente preenchendo nova entrega que dispara o fluxo.

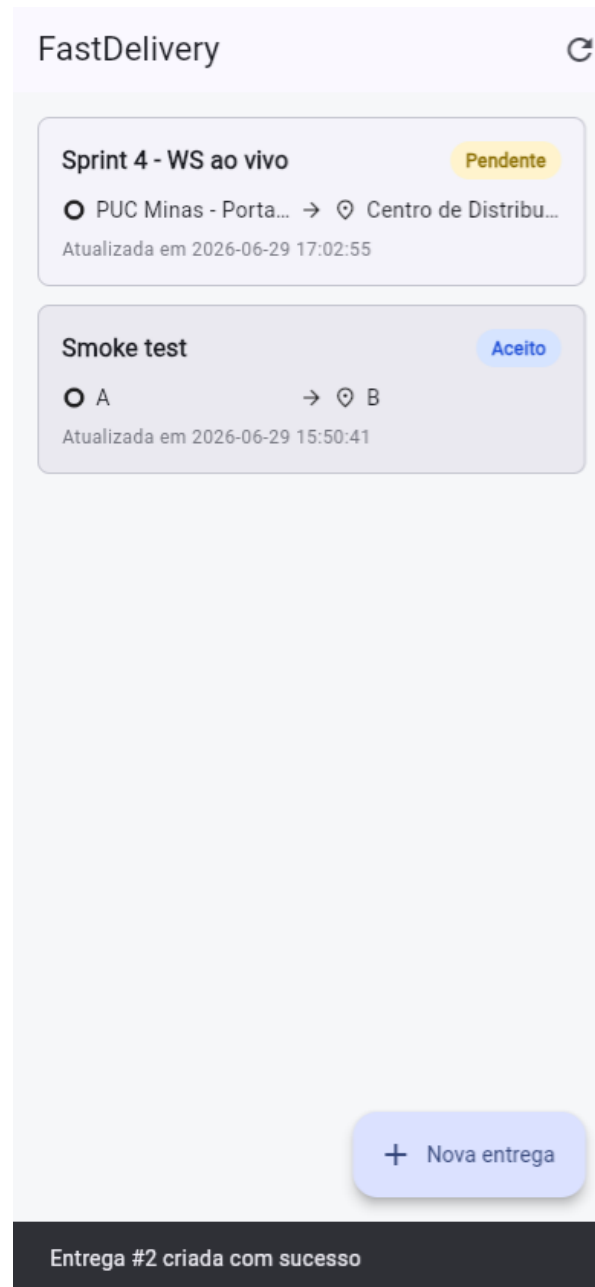


Figura 3 - Cliente apos criar a entrega #2 com status pendente.

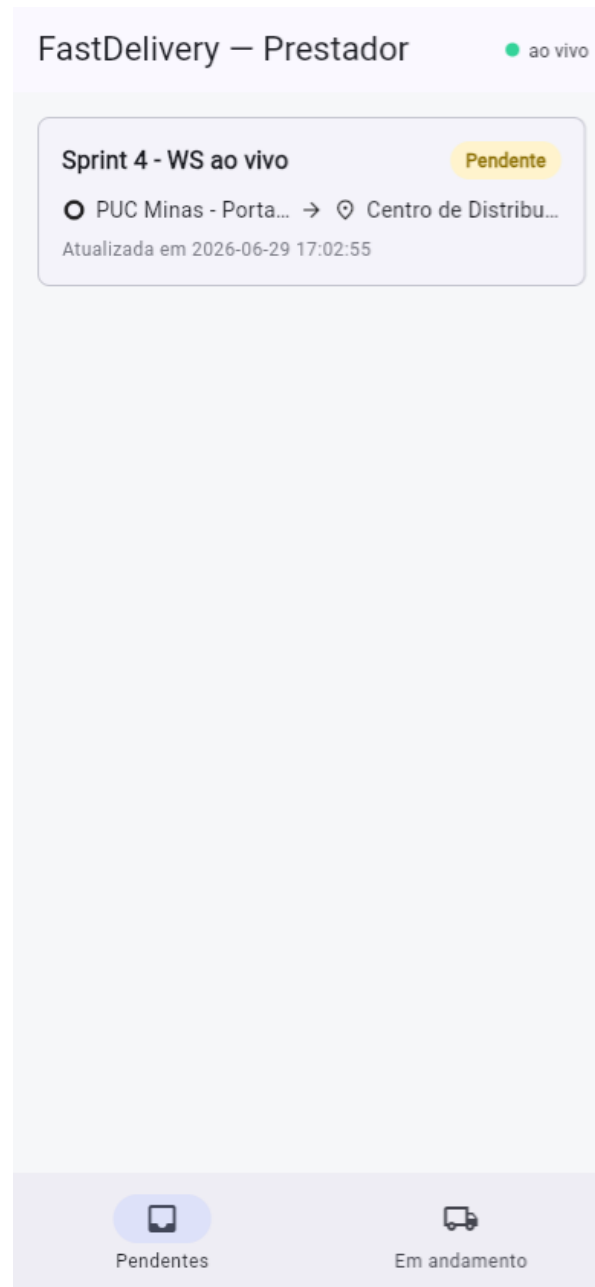


Figura 4 - Prestador recebeu a entrega #2 automaticamente em Pendentes.

 Solicitação #2

Sprint 4 - WS ao vivo

Pendente

☒ Pendente

☐ Aceito

☐ Em trânsito

☐ Concluído

 Origem

PUC Minas - Portaria 1

 Destino

Centro de Distribuicao FastDelivery

 Cliente

cliente-demo-001

 Criada em

2026-06-29 17:02:55

 Atualizada em

2026-06-29 17:02:55

 Recusar

 Aceitar

Figura 5 - Detalhe do prestador com ações Aceitar e Recusar.

← Solicitação #2

Sprint 4 - WS ao vivo

Aceito

☒ Pendente

☒ Aceito

☐ Em trânsito

☐ Concluído

☐ Origem

PUC Minas - Portaria 1

☐ Destino

Centro de Distribuicao FastDelivery

☐ Cliente

cliente-demo-001

☐ Criada em

2026-06-29 17:02:55

☐ Atualizada em

2026-06-29 17:03:44

→ Iniciar trânsito

Solicitação aceita

Figura 6 - Prestador aceita a entrega; status muda para aceito.

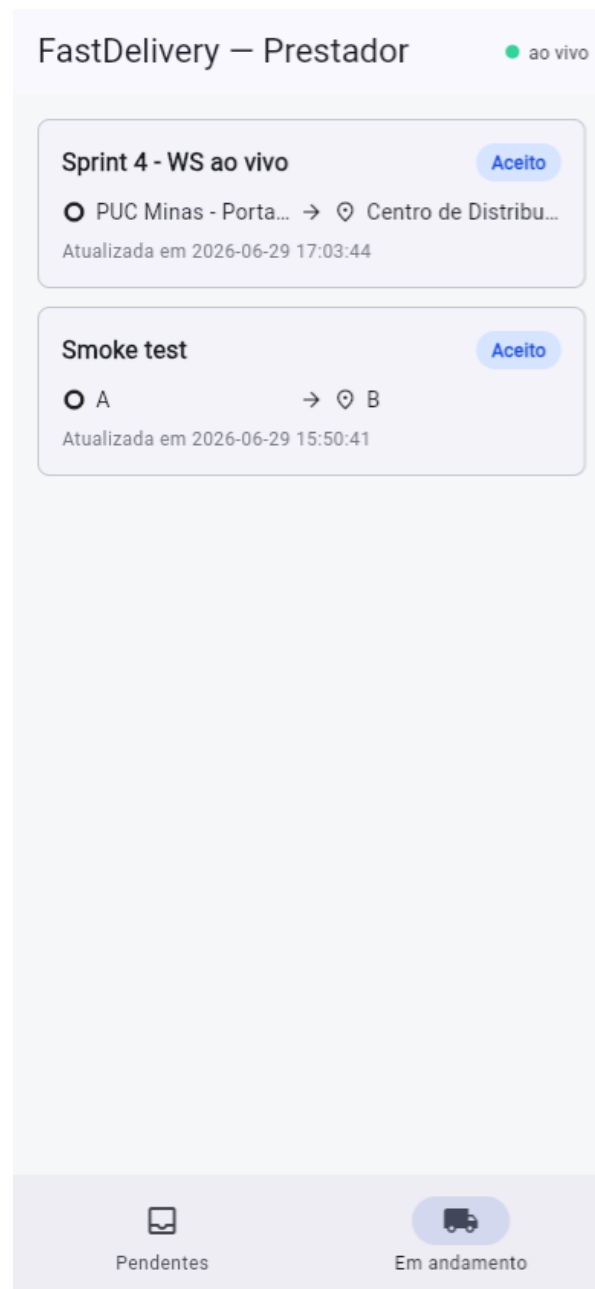


Figura 7 - Entrega aceita aparece na aba Em andamento.

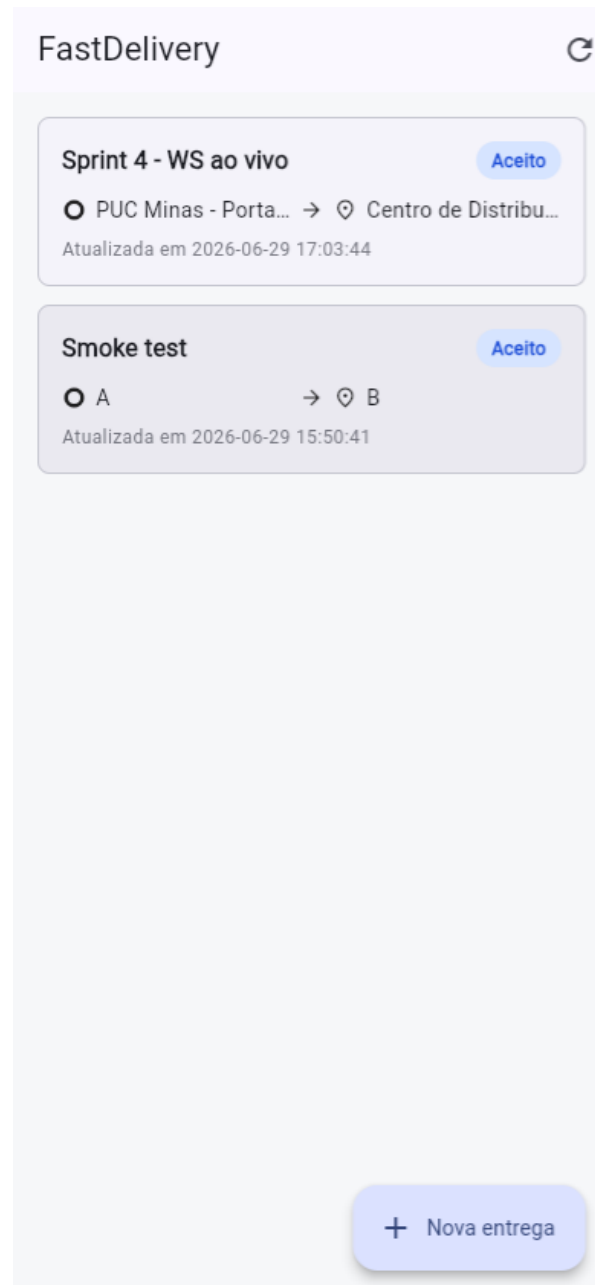


Figura 8 - Cliente reflete o status aceito por polling.

Refreshed 2026-06-29 14:04:56
Refresh every 5 seconds ▾
Virtual host All ▾
Cluster **rabbit@1eea7137b819**
User **guest** Log out

 3.13.7 Erlang 26.2.5.16

Overview Connections Channels
Exchanges **Queues and Streams** Admin

Displaying 3 items , page size up to: 100

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	fastdelivery.dlq	classic	D	running	0	0	0				
/	fastdelivery.entregas	classic	D DLX DLK	running	0	0	0	0.00/s	0.00/s	0.00/s	
/	fastdelivery.realtime	classic	AD	running	0	0	0	0.00/s	0.00/s	0.00/s	

► Add a new queue

HTTP API Documentation Tutorials
New releases Commercial edition
Commercial support Discussions Discord
Plugins GitHub

Figura 9 - RabbitMQ Management mostra filas do MOM, histórico e realtime.

Estilos de formatação ☐

```
[
  {
    "entrega_id": 2,
    "evento_id": "412a58dd-6226-48cb-ab63-36dc509bfd6b",
    "mensagem": "Nova entrega criada: Sprint 4 - WS ao vivo",
    "payload": {
      "cliente_id": "cliente-demo-001",
      "descricao": "Sprint 4 - WS ao vivo",
      "destino": "Centro de Distribuicao FastDelivery",
      "id": 2,
      "origem": "PUC Minas - Portaria 1",
      "status": "pendente"
    },
    "processado_em": "2026-06-29T14:02:55",
    "tipo": "entrega.criada"
  },
  {
    "entrega_id": 2,
    "evento_id": "3ae004cf-4a1d-42b5-8400-f8c30134fe44",
    "mensagem": "Status alterado: pendente \u2192 aceito",
    "payload": {
      "cliente_id": "cliente-demo-001",
      "id": 2,
      "status_anterior": "pendente",
      "status_novo": "aceito"
    },
    "processado_em": "2026-06-29T14:03:44",
    "tipo": "entrega.status_atualizado"
  }
]
```

Figura 10 - Endpoint GET /eventos com eventos processados pelo worker.

6. Validação técnica

Serviços executados:

- Docker Compose: container fastdelivery-rabbitmq em estado healthy, com porta AMQP 5672 e Management 15672.
- Backend: main.py na porta 5055, com producer RabbitMQ e ponte WebSocket /ws/eventos inicializada.
- Worker: consumer_worker.py em processo separado, consumindo a fila fastdelivery.entregas.
- Apps: builds web do cliente e do prestador servidos nas portas 9101 e 9102 e abertos no Google Chrome.

Comando/validação	Resultado
Backend unittest	Ran 27 tests in 1.231s - OK.
flutter analyze - cliente	No issues found.
flutter test - cliente	All tests passed - 15 testes.
flutter analyze - prestador	No issues found.

flutter test - prestador	All tests passed - 13 testes.
--------------------------	-------------------------------

O endpoint GET /eventos registrou os eventos entrega.criada e entrega.status_atualizado da entrega #2, confirmando que o producer publicou no RabbitMQ e que o consumer independente processou o histórico.

7. Dificuldades e soluções adotadas

Dificuldade	Solucao adotada
Evitar que prestador e worker disputem a mesma mensagem.	Criacao da fila dedicada fastdelivery.realtime ligada ao mesmo exchange topic.
Notificar mobile sem expor credenciais/broker RabbitMQ ao app.	Ponte no backend: consumidor RabbitMQ interno -> hub em memória -> WebSocket.
Manter consistência quando o WebSocket reconecta.	Controllers do prestador fazem re-sync REST ao conectar/reconectar.
Reutilizar contratos das sprints anteriores.	Ações do prestador usam PATCH /entregas/<id>/status com status já existentes.
Capturar evidências no Chrome para app projetado como mobile.	Build web local dos apps, API em 127.0.0.1 e Chrome de validação local.

8. Reflexão sobre os padrões estudados

Event-Driven Architecture aparece na forma como mudancas relevantes do domínio viram eventos: criar entrega publica entrega.criada e alterar status publica entrega.status_atualizado. A UI não depende de acoplamento direto entre cliente e prestador; ambos observam o estado do backend por mecanismos apropriados ao papel.

O MOM centraliza a comunicação indireta. RabbitMQ oferece exchange topic, filas, roteamento, ack e DLQ para a parte historica. A ponte WebSocket traduz esse modelo para o app do prestador, mantendo o broker privado e evitando que um app mobile tenha conexao AMQP direta.

A Clean Architecture foi aplicada com separacao clara entre presentation, application, domain, data e core nos apps Flutter, e entre controllers, use cases, repositories, domain e MOM no backend. Essa separacao reduziu o impacto da Sprint 4: a maior parte do trabalho entrou como adicao, sem quebrar os contratos REST das sprints anteriores.

REST continua adequado para comandos e consultas idempotentes ou pontuais: listar entregas, buscar detalhes e atualizar status. O WebSocket fica reservado para notificação assíncrona. Essa combinacao evita polling no prestador, mas preserva simplicidade no cliente.

9. Roteiro recomendado para o screencast

1. Mostrar RabbitMQ em execução no Docker e backend Flask com WS /ws/eventos.
2. Abrir app cliente e app prestador simultaneamente.
3. No prestador, mostrar indicador ao vivo e lista pendente vazia.
4. No cliente, criar uma entrega nova.
5. Mostrar a entrega aparecendo no prestador sem refresh manual.
6. Abrir detalhe no prestador e aceitar a entrega.
7. Mostrar a aba Em andamento no prestador e o cliente refletindo status aceito.
8. Encerrar mostrando GET /eventos ou RabbitMQ Management como evidência do MOM.

10. Conclusão

A Sprint 4 esta tecnicamente atendida para o escopo de app do prestador e integração final. O sistema demonstrou o ciclo cliente -> backend -> MOM -> prestador -> backend -> cliente, com notificação assíncrona real ao prestador e preservacao do histórico por consumer independente. A solucao também permanece evolutiva: autenticação, atribuição de prestador e CORS para uso web público podem ser adicionados futuramente sem reescrever o desenho central.

Referências

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

RICHARDSON, Chris. Microservices patterns: with examples in Java. Shelter Island: Manning, 2018.

COULOURIS, George et al. Distributed Systems: concepts and design. 5th ed. Boston: Addison-Wesley, 2011.

BAILEY, Thomas. Flutter for beginners. 3rd ed. Birmingham: Packt, 2023.

PUC MINAS. Projeto_LAMD_60445_2026_1.pdf. Laboratorio de Desenvolvimento de Aplicações Móveis e Distribuídas, 1º semestre de 2026.